

Statistique descriptive avec R

Correction TP 1

R possède des jeux de données intégrés. Nous allons travailler sur les données **iris**, une **data.frame** déjà présente dans R. On rappelle que pour accéder à l'aide d'une fonction, on peut utiliser **help(nom_fonction)**.

La fonction **str** appliquée à une **data.frame** permet de voir les noms des colonnes, leurs types, et quelques valeurs.

```
str(iris)
```

On peut avoir une représentation rapide des minimums, maximums, moyennes et quartiles de chaque colonne avec **summary**.

```
summary(iris)
```

Pour explorer au-delà d'un affichage dans la console, on peut utiliser la fonction **View**.

```
View(iris)
```

La fonction **tapply** est similaire à l'opération *Group By* de SQL. Elle a la forme **tapply(x, y, f)** où **x** est un vecteur de données quantitatives, **y** est un vecteur de modalités de même taille, et **f** est une fonction d'agrégation ($f : \mathbb{R}^n \rightarrow \mathbb{R}$). Ici, **tapply** calcule la fonction **f** sur **x** pour chaque modalité de **y**.

Par exemple, nous pouvons calculer la longueur moyenne de **Sepal.Length** dans **iris** pour chaque espèce.

```
tapply(iris$Sepal.Length, iris$Species, mean)
```

1. La fonction **f**, qui est ci-dessus **mean**, peut aussi être programmée. Calculez la moyenne des valeurs en excluant le minimum et le maximum, c'est-à-dire :

$$f(x) = \frac{1}{n-2} \left(\sum_{i=1}^n x_i - \min(x) - \max(x) \right).$$

```
f <- function(x)
  return((sum(x) - max(x) - min(x))/(length(x)-2))

tapply(iris$Sepal.Length, iris$Species, f)
```

2. Pour obtenir la matrice de corrélation des données quantitatives de `iris`, on exclut la dernière colonne et on utilise la fonction `cor`. Affichez la matrice de corrélation des données quantitatives de `iris`.

```
cor(iris[, -5])
```

3. La fonction `plot(x, y)` permet d'afficher `y` en fonction de `x`. Affichez `Petal.Length` en fonction de `Petal.Width`.

```
plot(iris$Petal.Width, iris$Petal.Length)
```

4. Changez de style de point en utilisant l'argument `pch=16`, et choisir une couleur *moderne* en piochant dans la palette <https://flatuicolors.com/palette/defo> (e.g. le bleu sombre).

```
plot(iris$Petal.Width, iris$Petal.Length, pch=16, col="#2980b9")
```

5. Mettez une étiquette pour les abscisses : `"Largeur des pétales"` et pour les ordonnées : `"Longueur des pétales"` avec les arguments `xlab` et `ylab` de `plot`.

```
plot(iris$Petal.Width, iris$Petal.Length, pch=16, col="#2980b9",  
     xlab="Largeur des pétales", ylab="Longueur des pétales")
```

6. Pour faire une régression linéaire, on utilise la syntaxe `lm(y ~ x, data=mес_donnees)` où `mес_donnees` est la `data.frame` qui contient les données : `x` est le nom de la colonne de la variable explicative, `y` est le nom de la colonne de la variable à expliquer. Faites la régression linéaire de `Petal.Width` en fonction de `Petal.Length` et enregistrez-la dans une variable nommée `reg`.

```
(reg <- lm(Petal.Length ~ Petal.Width, data=iris))
```

7. Vérifiez que c'est une liste et obtenez le nom des éléments avec `names`.

```
is.list(reg)  
names(reg)
```

8. Ajoutez la droite de régression avec `abline` avec une épaisseur de 2 (argument `lwd`) et une couleur sombre.

```
plot(iris$Petal.Width, iris$Petal.Length, pch=16, col="#2980b9",  
     xlab="Largeur des pétales", ylab="Longueur des pétales")  
abline(reg, lwd=2, col="#2c3e50")
```

9. Ajoutez une légende avec `legend` en haut à gauche. Indiquer le type de point (argument `pch`) dans la légende affiche un point, et indiquer l'épaisseur du trait (argument `lwd`) affiche un trait. `NA` permet de dire qu'il n'y a rien. La fonction `legend` est vectorielle.

```
plot(iris$Petal.Width, iris$Petal.Length, pch=16, col="#2980b9",  
     xlab="Largeur des pétales", ylab="Longueur des pétales")  
abline(reg, lwd=2, col="#2c3e50")  
legend("topleft", legend=c("Données", "Régression"), pch=c(16, NA),  
      lwd = c(NA, 2), col = c("#2980b9", "#2c3e50"))
```

10. Dans le graphique, les largeurs et longueurs des pétales sont affichées pour toutes les espèces. On souhaite mettre une couleur différente en fonction de l'espèce. On choisit la couleur "#16a085" pour **setosa**, "#c0392b" pour **versicolor** et "#8e44ad" pour **virginica**. Construisez un vecteur **couleurs** de la même longueur que le nombre de lignes des données, avec la valeur constante "#16a085".

```
couleurs ← rep("#16a085", nrow(iris))
```

11. Pour les indices où l'espèce est **versicolor**, remplacez la valeur de **couleurs** par "#c0392b". Faites de même pour **virginica**.

```
couleurs[iris$Species == "versicolor"] ← "#c0392b"
couleurs[iris$Species == "virginica"] ← "#8e44ad"
```

12. Affichez le graphique complet avec les couleurs par espèce, en adaptant la légende.

```
plot(iris$Petal.Width, iris$Petal.Length, pch=16, col=couleurs,
     xlab="Largeur des pétales", ylab="Longueur des pétales")
abline(reg, lwd=2, col="#2c3e50")
legend("topleft", legend=c("Setosa", "Versicolor", "Virginica", "Régression"),
      pch=c(16, 16, 16, NA), lwd = c(NA, NA, NA, 2), col =
      c("#16a085", "#c0392b", "#8e44ad", "#2c3e50"))
```

13. Ajoutez un maillage avec la fonction **grid** et mettez une couleur de fond à votre légende.

```
plot(iris$Petal.Width, iris$Petal.Length, pch=16, col=couleurs,
     xlab="Largeur des pétales", ylab="Longueur des pétales")
grid(col = "#bdc3c7", lty = 3)
abline(reg, lwd=2, col="#2c3e50")
legend("topleft", legend=c("Setosa", "Versicolor", "Virginica", "Régression"),
      pch=c(16, 16, 16, NA), lwd = c(NA, NA, NA, 2), col =
      c("#16a085", "#c0392b", "#8e44ad", "#2c3e50"), bg="#ecf0f1")
```

14. Enregistrez votre graphique au format PDF grâce à la fonction **pdf** avec une largeur de 10 et une hauteur de 7. On place l'appel à la fonction **pdf** avant de générer le graphique, et une fois le graphique terminé, on conclut avec **dev.off()** qui ferme et enregistre le PDF.

```
pdf("mon_graphique.pdf", width=10, height=7)
plot(iris$Petal.Width, iris$Petal.Length, pch=16, col=couleurs,
     xlab="Largeur des pétales", ylab="Longueur des pétales")
grid(col = "#bdc3c7", lty = 3)
abline(reg, lwd=2, col="#2c3e50")
legend("topleft", legend=c("Setosa", "Versicolor", "Virginica", "Régression"),
      pch=c(16, 16, 16, NA), lwd = c(NA, NA, NA, 2), col =
      c("#16a085", "#c0392b", "#8e44ad", "#2c3e50"), bg="#ecf0f1")
dev.off()
```

Statistique descriptive avec R

TP 2

Téléchargez les données au format Excel disponibles à l'adresse suivante :

<https://www.insee.fr/fr/statistiques/4268033>. Dans l'onglet *Figure 2*, copiez le tableau (à partir de la cellule A4) dans un fichier texte et enregistrez-le sous "inflation.csv".

1. Identifiez le séparateur de données et le séparateur décimal. Comment spécifier ce dernier dans R ?

Le séparateur de données est la tabulation, dans R ce caractère est encodé avec "\t". Le séparateur décimal est la virgule ",".

2. Importez les données avec la fonction `read.csv`.

```
inflation <- read.csv("inflation.csv", sep="\t", dec=",")
```

3. Vous pourriez rencontrer des problèmes avec les accents et autres caractères spéciaux, dus à un encodage de fichier différent de celui attendu par la fonction d'importation. L'encodage est généralement "UTF-8". Précisez-le.

```
inflation <- read.csv("inflation.csv", sep="\t", dec=",",  
  fileEncoding="UTF-8")
```

4. Ajoutez un argument `colClasses` pour préciser le type des colonnes dès l'importation. Vous pouvez tester les erreurs générées si vous spécifiez mal le séparateur décimal.

```
inflation <- read.csv("inflation.csv", sep="\t", dec=",",  
  fileEncoding="UTF-8", colClasses = c("character", rep("numeric",  
  5))) # Pour tester, changez dec=", " en dec=".". "
```

5. Importez à nouveau les données mais avec la fonction `fread` du paquet `data.table`, en précisant l'encodage.

```
library(data.table)  
inflation <- fread("inflation.csv", sep="\t", dec=",", encoding="UTF-  
  8", colClasses = c("character", rep("numeric", 5)))
```

6. Passez vos données en `data.frame` et revenez en `data.table` sans copie.

```
setDF(inflation)  
setDT(inflation)
```

7. Transformez la première colonne au format `Date`, en fixant les dates au 1er janvier..

```
inflation$Mois ← as.Date(paste0(inflation$Mois, "-01"), format = "%Y-%m-%d")
```

8. Vos données sont de la plus récente à la plus ancienne. Triez-les de la plus ancienne à la plus récente à l'aide de la fonction `order`.

```
inflation ← inflation[order(Mois)]  
# En data.frame, il faut utiliser le code suivant (compatible et  
# valable avec data.table) :  
inflation ← inflation[order(inflation$Mois), ]
```

9. Avec `matplot`, en utilisant comme premier argument le vecteur des dates en abscisse, et en second argument la `data.table` sans la colonne des dates, affichez tous les indices d'inflation.

```
matplot(inflation$Mois, inflation[, -1], xlab = "Mois", ylab = "  
Indice des prix à la consommation", type="l", lty=1, lwd=2, col=  
c("#c0392b", "#bdc3c7", "#f39c12", "#2980b9", "#2c3e50"))
```

10. Ajoutez la légende en haut à gauche.

```
matplot(inflation$Mois, inflation[, -1], xlab = "Mois", ylab = "  
Indice des prix à la consommation", type="l", lty=1, lwd=2, col=  
c("#c0392b", "#bdc3c7", "#f39c12", "#2980b9", "#2c3e50"))  
legend("topleft", legend = colnames(inflation)[-1], lty=1, lwd=2,  
col=c("#c0392b", "#bdc3c7", "#f39c12", "#2980b9", "#2c3e50"), bg  
="#ecf0f1")
```

11. Les indices sont en base 100 pour l'année 2015. Mettez-les en base 100 par rapport à la première date des données. Rappel : avec des `data.frame` on peut accéder à la colonne `i` avec `inflation[, i]` mais en `data.table`, par défaut, il interprète `i` comme un nom de colonne sans l'évaluer, comportement similaire à `inflation$i`. Pour l'évaluer, il faut soit utiliser la notation des listes : `inflation[[i]]`, soit forcer l'évaluation de `i` avec `inflation[, i, with = FALSE]`.

```
for(i in 2:ncol(inflation))  
  inflation[[i]] ← inflation[[i]]/inflation[[i]][1]*100  
  
matplot(inflation$Mois, inflation[, -1], xlab = "Mois", ylab = "  
Indice des prix à la consommation", type="l", lty=1, lwd=2, col=  
c("#c0392b", "#bdc3c7", "#f39c12", "#2980b9", "#2c3e50"))  
legend("topleft", legend = colnames(inflation)[-1], lty=1, lwd=2,  
col=c("#c0392b", "#bdc3c7", "#f39c12", "#2980b9", "#2c3e50"), bg  
="#ecf0f1")
```

Remarque. Pour des données de petite taille, le paquet `data.table` est superflu en termes d'efficacité; c'est davantage un choix pour sa syntaxe flexible. Cependant, pour des données importantes ou des manipulations complexes, il est plus efficace et rapide.

Statistique descriptive avec R

TP 2

Téléchargez au format CSV l'historique d'un portefeuille investi sur le SP500 :
<https://curvo.eu/backtest/fr/indice/sp-500?currency=eur#graphique>.

1. Importez les données et mettez la première colonne au format **Date** au 1er janvier, vous spécifierez le type des données à l'importation.

```
sp500 ← read.csv("chart.csv", colClasses=c("character", "numeric"),  
  sep=";", dec=".")  
sp500$Date ← as.Date(paste0("01/", sp500$Date), format="%d/%m/%Y")
```

2. Ajoutez la colonne **Perf** définie comme la performance mensuelle du SP500 :

$$\text{Perf}_t := \frac{\text{SP500}_t - \text{SP500}_{t-1}}{\text{SP500}_{t-1}}, \quad t \geq 2.$$

Mettez **NA** pour le premier mois où ce n'est pas calculable.

```
n ← nrow(sp500)  
sp500$Perf ← c(NA, (sp500$S.P.500[-1] - sp500$S.P.500[-n])/  
  sp500$S.P.500[-n])
```

3. Affichez l'histogramme des performances.

```
hist(sp500$Perf, col="#3498db", xlab="Performance mensuelle", ylab=  
  "Fréquence", main="SP500")
```

4. Définissez une variable **Perf** : le vecteur des performances calculées sans la donnée manquante, puis définissez une variable **batons** : une séquence qui démarre de la valeur minimale de vos données arrondie au centième et s'achève à la valeur maximale arrondie au centième, avec un pas d'un centième.

```
Perf ← sp500$Perf[!is.na(sp500$Perf)]  
batons ← seq(round(min(Perf), 2), round(max(Perf), 2), by = 0.01)
```

5. Affichez l'histogramme en utilisant un pas d'un centième pour les bâtons.

```
hist(Perf, breaks = batons, col="#3498db", xlab="Performance  
  mensuelle", ylab="Fréquence", main="SP500")
```

6. Calculez la moyenne et l'écart-type des performances.

```
mu ← mean(Perf)
sigma ← sd(Perf)
```

7. Sur votre histogramme, affichez la densité gaussienne avec la moyenne et la variance empiriques calculées. Pour comparer l'histogramme à la densité, il faut ajouter l'argument `freq=FALSE` à celui-ci.

```
hist(Perf, breaks = batons, col="#3498db", freq = FALSE, xlab="
  Performance mensuelle", ylab="Densité", main="SP500")
lines(batons, dnorm(batons, mu, sigma), col="#e74c3c", lwd=2)
```

8. La fonction `density` permet d'obtenir la densité empirique par méthode de noyaux définie par :

$$\hat{f}(x) := \frac{1}{n} \sum_{i=1}^n \frac{1}{h} K\left(\frac{x - x_i}{h}\right), \quad x \in \mathbb{R},$$

où $(x_i)_{1 \leq i \leq n}$ est l'échantillon observé, h est la *fenêtre* (`R` la calculera de manière optimale) et K est une densité centrée réduite (par défaut, `R` utilise la loi normale).

En utilisant les paramètres par défaut de `density`, affichez la densité empirique des performances et comparez à celle de la loi normale.

```
plot(density(Perf), col="#3498db", xlab="Performance mensuelle",
  ylab="Densité", main="SP500", lwd=2)
lines(batons, dnorm(batons, mu, sigma), col="#c0392b", lwd=2)
```

9. Nous allons tester si les données sont normales. Nous allons utiliser le test de Jarque-Bera. Calculez le skewness et le kurtosis et commentez par rapport à la loi normale.

```
S ← mean(((sp500$Perf - mu)/sigma)^3)
K ← mean(((sp500$Perf - mu)/sigma)^4)
# Loi normale S=0 : asymétrie à gauche,
# et K=3 : queues plus épaisses.
```

10. Le test de Jarque-Bera repose sur le résultat de convergence suivant : si les données $(X_i)_{1 \leq i \leq n}$ sont normales et i.i.d.,

$$JB_n := n \left(\frac{S_n^2}{6} + \frac{(K_n - 3)^2}{24} \right) \xrightarrow{n \rightarrow +\infty} \chi_2^2,$$

où S_n est le skewness des données et K_n en est le kurtosis. Calculez JB_n .

```
n_Perf ← length(Perf)
JB ← n_Perf*(S^2/6 + (K-3)^2/24)
```

11. La p-valeur est la probabilité, sous l'hypothèse nulle $\{H_0 : \text{Les performances sont gaussiennes}\}$, d'obtenir une statistique de test au moins aussi extrême que celle observée, dans la queue de la distribution. Ici, pour le test de Jarque-Bera, elle correspond à $\mathbb{P}(\chi_2^2 > JB_n)$ où JB_n est la valeur du test observée empiriquement. Calculez la p-valeur et concluez.

```
pvaleur ← 1-pchisq(JB, df=2)
```

12. Mettez le calcul du test dans une fonction de [R](#).

```
Test.JB ← function(x)
{
  z ← x[!is.na(x)]
  n_z ← length(z)
  mu ← mean(z)
  sigma ← sd(z)
  S ← mean(((z - mu)/sigma)^3)
  K ← mean(((z - mu)/sigma)^4)
  JB ← n_z*(S^2/6 + (K-3)^2/24)
  return(1-pchisq(JB, df=2))
}
Test.JB(sp500$Perf)
```


Statistique descriptive avec R

Correction TP 4

Nous allons travailler sur les données **airquality** intégrées de **R**.

1. Explorez vos données.

```
str(airquality)
summary(airquality)
View(airquality)
```

2. La colonne **Day** donne le jour et la colonne **Month** donne le mois. En consultant l'aide de **airquality**, ajoutez une colonne **Date** qui est la date au format **Date** de **R**. Puis supprimez les colonnes **Day** et **Month**.

```
airquality$Date <- as.Date(paste("1973", airquality$Month,
  airquality$Day, sep = "-")) # Ou autre format précisé
airquality$Day <- NULL
airquality$Month <- NULL
```

3. La fonction **par** permet de contrôler les paramètres globaux du graphique. L'argument **mfrow** = **c(n, m)** permet d'afficher plusieurs graphiques à la fois : il y aura n lignes et m colonnes, c'est-à-dire $n \times m$ graphiques. L'argument **mar** contrôle la taille des marges. Appelez **par** dans le but d'afficher 4 graphiques avec comme marge **c(4.1, 4, 2, 2)**, puis affichez les vecteurs **Ozone**, **Solar.R**, **Wind** et **Temp** en fonction de **Date**, en utilisant un **pch** de 16 et des couleurs modernes différentes pour chaque graphique.

```
par(mfrow=c(2,2), mar=c(4.1, 4, 2, 2))
plot(airquality$Date, airquality$Ozone, xlab="", , ylab="Ozone",
  col="#c0392b", pch=16)
plot(airquality$Date, airquality$Solar.R, xlab="", ylab="Solar.R",
  col="#27ae60", pch=16)
plot(airquality$Date, airquality$Wind, xlab="", ylab="Wind", col="
  #2980b9", pch=16)
plot(airquality$Date, airquality$Temp, xlab="", ylab="Temp", col="
  #2c3e50", pch=16)
```

4. En explorant les données, nous avons pu voir la présence de **NA**. Calculez la proportion de **NA** par colonnes, ce qui permet d'identifier quelles colonnes sont touchées et en quelle proportion.

```
colMeans(is.na(airquality))
```

5. Nous souhaitons estimer la valeur des données manquantes. Affichez **Ozone** et **Temp** en fonction de **Wind** et de **Temp** (4 graphiques) et commentez.

```
par(mfrow=c(2,2), mar=c(4.1, 4, 2, 2))
plot(airquality$Wind, airquality$Ozone, xlab="Wind", ylab="Ozone",
     col="#c0392b", pch=16)
plot(airquality$Temp, airquality$Ozone, xlab="Temp", ylab="Ozone",
     col="#c0392b", pch=16)
plot(airquality$Wind, airquality$Solar.R, xlab="Wind", ylab="
     Solar.R", col="#27ae60", pch=16)
plot(airquality$Temp, airquality$Solar.R, xlab="Temp", ylab="
     Solar.R", col="#27ae60", pch=16)
```

6. Nous allons utiliser une régression linéaire afin d'estimer les données manquantes. Faites la régression linéaire de **Ozone** et de **Solar.R** contre toutes les autres variables et, avec **summary**, itérativement, supprimez les variables explicatives non significatives (on utilisera le seuil de 5%).

```
summary(lm(Ozone ~ ., data=airquality))
summary(lm(Ozone ~ . - Date, data=airquality))

summary(lm(Solar.R ~ ., data=airquality))
summary(lm(Solar.R ~ . - Wind, data=airquality))
summary(lm(Solar.R ~ . - Wind - Temp, data=airquality))
summary(lm(Solar.R ~ Ozone, data=airquality))
```

7. On observe que **Ozone** et **Solar.R** s'utilisent mutuellement pour s'estimer. Affichez **Ozone** en fonction de **Solar.R**

```
par(mfrow=c(1,1))
plot(airquality$Solar.R, airquality$Ozone, xlab="Solar.R", ylab="
     Ozone", col="#c0392b", pch=16)
```

8. Est-ce qu'il y a des cas où à la fois **Ozone** et **Solar.R** sont **NA**? Ces données ne pourront pas être estimées dans un premier temps.

```
sum(is.na(airquality$Ozone) & is.na(airquality$Solar.R))
```

9. Faites une copie de **airquality** dans une variable nommée **aq**, faites la régression linéaire sélectionnée pour la variable expliquée **Ozone**, puis remplacez dans **aq** les données où **Ozone** est **NA** par l'estimation de la régression linéaire, seulement quand **Solar.R** n'est pas **NA**. On utilisera **predict** pour obtenir les estimations de **Ozone** par la régression linéaire.

```
aq <- airquality
reg_Ozone <- lm(Ozone ~ . - Date, data = airquality)
Indices <- is.na(airquality$Ozone) & !is.na(airquality$Solar.R)
aq$Ozone[Indices] <- predict(reg_Ozone, newdata = airquality[Indices
, ])
aq$Ozone_isna <- Indices
```

10. Faites de même pour remplacer données manquantes de **Solar.R** là où c'est possible.

```
reg_Solar.R <- lm(Solar.R ~ Ozone, data=airquality)
Indices <- is.na(airquality$Solar.R) & !is.na(airquality$Ozone)
aq$Solar.R[Indices] <- predict(reg_Solar.R, newdata = airquality[
  Indices, ])
aq$Solar.R_isna <- Indices
```

11. Affichez **Ozone** et **Solar.R** en fonction de **Date** avec les données complétées. Les données estimées seront mises en évidence, dans les deux graphiques, elles auront un **pch** de 17 et une couleur sombre. Faites une critique.

```
par(mfrow=c(2,1))
plot(aq$Date, aq$Ozone, xlab="", , ylab="Ozone", pch=16+
  aq$Ozone_isna, col=c("#c0392b", "#2c3e50")[1+aq$Ozone_isna])
plot(aq$Date, aq$Solar.R, xlab="", ylab="Solar.R", pch=16+
  aq$Solar.R_isna, col=c("#27ae60", "#2c3e50")[1+aq$Solar.R_isna])
# On observe une donnée estimée négative pour Ozone ce qui est
# impossible.
# Nous aurions pu estimer log(Ozone) contre un modèle linéaire,
# ce qui aurait donné Ozone = exp(Linéaire) et voir si les ré
# sultats étaient meilleurs.
```

Statistique descriptive avec R

Correction TP 5

Nous allons travailler sur les données américaines **flights14** du Bureau des statistiques des transports (*Bureau of Transportation Statistics*) pour les vols du départ de New York de janvier à octobre 2014.

1. Importez avec le paquet **data.table** et explorez vos données.

```
library(data.table)
flights <- fread("https://raw.githubusercontent.com/Rdatatable/
data.table/master/vignettes/flights14.csv", header=TRUE, sep=",",
, encoding="UTF-8")
```

2. Il ne faut jamais parcourir les lignes une à une dans un **data.table**, et de même dans un **data.frame**. La fonction **system.time** permet de mesurer le temps de calcul effectif d'un bloc de code. Exécutez les deux codes suivants. Testez également avec l'objet en **data.frame** (ne pas oublier de vous remettre en **data.table** pour la suite). Comparez à une solution efficace que vous proposerez.

```
# Code 1 avec extraction de ligne puis sélection
system.time({
  s = 0
  for(i in 1:nrow(flights))
    if(flights[i, ]$origin == "JFK")
      s = s + flights[i, ]$dep_delay
})

# Code 2 avec extraction de ligne et sélection
system.time({
  s = 0
  for(i in 1:nrow(flights))
    if(flights[i, "origin"] == "JFK")
      s = s + flights[i, "dep_delay"]
})
```

Sur une machine personnelle particulière, le premier s'exécute en environ 50 secondes et le second en environ 100 secondes. Cela s'explique car à chaque ligne, il construit un **data.table** si on regarde en détail, ce qui est lourd étant donné l'architecture en liste de vecteurs de l'objet. Cette lenteur a aussi lieu pour le cas où on extrait qu'un élément directement (Code 2). En **data.frame**, en faisant **setDF(flights)** avant d'exécuter à nouveau le code, on obtient respectivement 35 secondes et 5 secondes. Le dernier est beaucoup plus rapide car en sélectionnant un seul élément, il ne construit pas une **data.frame**. L'objet **data.table** est en général

plus rapide que `data.frame`, mais il n'est pas adapté à l'extraction sur une seule ligne (comme `data.frame`). Les codes suivants sont deux exemples pour résoudre le problème de manière efficace, le premier est un code classique `data.frame` compatible avec les deux formats, le second est spécifique aux `data.table`. Leur temps d'exécution est quasi instantané (environ 2 millisecondes), le facteur de gain est de l'ordre de grandeur de 10 000 à 100 000. Conclusion : ne jamais parcourir les lignes.

```
# Extraction des colonnes dans lesquelles on filtre
sum(flights$dep_delay[flights$origin == "JFK"])
# Rappel : on peut directement donner le nom des colonnes : ce ne
# sont pas des variables évaluées par défaut. Dans l'indice des
# colonnes on peut préciser à la source qu'on en veut la somme.
flights[origin == "JFK", sum(dep_delay)]
```

3. Récupérez la moyenne et l'écart-type du retard au départ en utilisant la fonction `tapply`.

```
# Séparément
tapply(flights$dep_delay, flights$origin, mean)
tapply(flights$dep_delay, flights$origin, sd)
# De manière jointe
mean_sd <- function(x)
  return(c(Moyenne = mean(x), EcartType = sd(x)))
tapply(flights$dep_delay, flights$origin, mean_sd)
```

4. Faites la question précédente en utilisant la syntaxe de `data.table`. On utilisera la syntaxe `mon.data.table[, f(colonneB), by=colonneA]` où on agrège par `f` la `colonneB` selon les modalités de la `colonneA`. Pour obtenir ensemble la moyenne et l'écart-type (mais aussi pour nommer une colonne au renvoi), on utilisera la syntaxe plus générale `mon.data.table[, .(nom1 = f(colonneB), nom2 = g(colonneC)), by=colonneA]`.

```
flights[, mean(dep_delay), by=origin]
flights[, sd(dep_delay), by=origin]
flights[, .(Moyenne = mean(dep_delay), EcartType = sd(dep_delay)),
  by=origin]
```

5. On rappelle que si les $(X_i)_{1 \leq i \leq n}$ sont des variables aléatoires i.i.d. et de carré intégrable, nous avons le théorème central limite sur $\bar{X}_n$ la moyenne empirique avec $\hat{\sigma}_n$ l'écart-type empirique :

$$\sqrt{n} \frac{\bar{X}_n - \mu}{\hat{\sigma}_n} \xrightarrow[n \rightarrow +\infty]{loi} \mathcal{N}(0, 1),$$

ce qui permet de construire un intervalle de confiance asymptotique :

$$\mathbb{P} \left(\mathbb{E}(X) \in \left[\bar{X}_n \pm q_{1-\frac{\alpha}{2}} \frac{\hat{\sigma}_n}{\sqrt{n}} \right] \right) \xrightarrow[n \rightarrow +\infty]{} 1 - \alpha,$$

où $q_{1-\frac{\alpha}{2}}$ est le quantile d'ordre $1 - \frac{\alpha}{2}$ d'une loi normale centrée réduite. Pour chaque aéroport de départ donnez la moyenne et les deux bornes de l'intervalle de confiance au niveau de 95% en exploitant la syntaxe de `data.table` et commentez.

```
flights_ic ← flights[, .(Moyenne = mean(dep_delay), EcartType = sd(
  dep_delay), N = length(dep_delay)), by=origin]
flights_ic[, .(origin, Moyenne, IC_inf = Moyenne - qnorm(0.975)*
  EcartType/sqrt(N), IC_sup = Moyenne + qnorm(0.975)*EcartType/
  sqrt(N))]
```

Le code ci-dessus aurait pu se faire en une ligne.
A priori, sans faire rigoureusement le test, EWR est en moyenne
significativement plus en retard et LGA l'est moins (IC
disjoints).

6. Construisez un vecteur de couleurs où à chaque aéroport de départ on associe une couleur.

```
# Solution 1
col ← rep("#2980b9", nrow(flights)) # EWR
col[flights$origin == "JFK"] ← "#27ae60"
col[flights$origin == "LGA"] ← "#c0392b"
```

```
# Solution 2
colors ← c("EWR" = "#2980b9", "JFK" = "#27ae60", "LGA" = "#c0392b")
col ← colors[flights$origin]
```

La suite est faite avec la solution 2, on aura la variable
colors.

7. Affichez `arr_delay` en fonction de `dep_delay` en mettant une couleur différente en fonction de l'aéroport de départ. Ajoutez une légende. Si vous enregistrez le graphique, en quel format est-ce préférable ?

```
plot(flights$dep_delay, flights$arr_delay, pch=16, col=col, xlab="
  Retard au départ", ylab="Retard à l'arrivée")
legend("topleft", names(colors), col=colors, pch=16)
```

Compte tenu du nombre de points, le format pdf (vectoriel)
conduit à un graphique lourd et long à afficher dans un document
, il est préférable de l'enregistrer en png avec une bonne ré
solution. Sinon il faut mettre moins de points, comme un sous-é
chantillon, ou visualiser autrement.

8. Calculez la corrélation linéaire entre `dep_delay` et `arr_delay` en fonction de l'aéroport d'origine. Puis calculez la corrélation entre `dep_delay` et `arr_delay - dep_delay` (qui est le retard ou l'avance additionnelle après départ) en fonction de l'origine et pour les données complètes. Que remarquez-vous ?

```
flights[, cor(dep_delay, arr_delay), by= origin]
flights[, cor(dep_delay, arr_delay - dep_delay), by= origin]
flights[, cor(dep_delay, arr_delay - dep_delay)]
```

Le retard à l'arrivée est naturellement très corrélé au retard au
départ, mais la différence ne l'est plus ou très peu.

9. Affichez le graphique précédent, mais en comparant cette fois-ci à `arr_delay - dep_delay` à `dep_delay`.

```
plot(flights$dep_delay, flights$arr_delay-flights$dep_delay, pch
     =16, col=col, xlab="Retard au départ", ylab="Retard à l'arrivée"
     )
legend("topright", names(colors), col=colors, pch=16)
```

10. Ajoutez une colonne `dep_delay_group` qui vaut "`<=60`" si le retard au départ est inférieur à 60, "`>60&<=300`" s'il est entre 60 et 180 et "`>300`" s'il est supérieur à 180.

```
dep_delay_group_name <- c("<=60", ">60&<=300", ">300")
dep_delay_group <- dep_delay_group_name[(flights$dep_delay > 180) +
    (flights$dep_delay > 60) + 1]
flights$dep_delay_group <- dep_delay_group
# ou flights <- flights[, dep_delay_group := dep_delay_group]
```

11. Pour `arr_delay - dep_delay`, calculez la moyenne, l'écart-type, la corrélation linéaire en fonction de `arr_delay - dep_delay`, et le nombre de données pour chaque `dep_delay_group_name`.

```
flights[, .(Moyenne = mean(arr_delay - dep_delay), EcartType = sd(
    arr_delay - dep_delay), Cor = cor(dep_delay, arr_delay -
    dep_delay), N = length(dep_delay)), by= dep_delay_group]
```

12. Affichez `arr_delay - dep_delay` en fonction de `dep_delay` avec une couleur différente par `dep_delay_group` et commentez.

```
colors <- c("#2980b9", "#27ae60", "#c0392b")
names(colors) <- dep_delay_group_name
col <- colors[flights$dep_delay_group]

plot(flights$dep_delay, flights$arr_delay-flights$dep_delay, pch
     =16, col=col, xlab="Retard au départ", ylab="Retard à l'arrivée"
     )
legend("topright", names(colors), col=colors, pch=16)
# On a l'impression que plus dep_delay est élevé, moins il y a de
# variabilité. Or ce n'est pas ce qu'on observe dans les données (
# cf question précédente). C'est un effet de taille de chaque
# classe. L'effectif moyen diminue par classe avec dep_delay_group
# , donc l'occurrence de valeurs extrêmes aussi.
```

Statistique descriptive avec R

TP 6

Correction

Nous allons travailler sur des données d'assurance du paquet **CASdatasets**.

1. Le paquet est trop volumineux pour être hébergé sur le CRAN en raison de données lourdes (236 Mo). Commencez par l'installer à l'aide du code suivant, puis vérifiez qu'il se charge correctement.

```
install.packages("CASdatasets", repos = "https://cas.uqam.ca/pub/",  
  type="source")  
library(CASdatasets)
```

2. Chargez les jeux de données **freMTPLfreq** et **freMTPLsev** avec la fonction **data**, puis convertissez-les en objets **data.table** sans effectuer de copie. Le premier jeu de données décrit les polices d'assurance automobile, avec leurs caractéristiques et le nombre de sinistres associés ; le second contient les détails des sinistres individuels correspondants.

```
library(data.table)  
data(freMTPLfreq)  
data(freMTPLsev)  
setDT(freMTPLfreq)  
setDT(freMTPLsev)
```

3. Commençons par le jeu de données **freMTPLfreq**. La colonne **ClaimNb** indique le nombre de sinistres observés par police. Affichez le tableau des fréquences pour chaque nombre de sinistres, puis calculez la moyenne et la variance de cette variable.

```
table(freMTPLfreq$ClaimNb)  
mean(freMTPLfreq$ClaimNb)  
var(freMTPLfreq$ClaimNb)
```

4. La colonne **Exposure** correspond à la durée d'observation de chaque assuré (en années). Pour un assuré i d'exposition E_i , on s'attend à λ sinistres par unité d'exposition, soit $\mathbb{E}(N_i) = \lambda E_i$ où N_i est le nombre de sinistres. Sous une loi de Poisson $N_i \sim \mathcal{P}(\lambda E_i)$, on aurait aussi $\text{Var}(N_i) = \lambda E_i$.

Proposez deux estimateurs de λ : $\hat{\lambda}_n^1$ basé sur la moyenne empirique (moyenne pondérée) et $\hat{\lambda}_n^2$ basé sur la variance (sous hypothèse poissonnienne). Concluez, sans test formel, si les données semblent suivre a priori une loi de Poisson.

Soit n le nombre de polices. D'après $\mathbb{E}(N_i/E_i) = \lambda$, un estimateur simple (non pondéré) serait

$$\hat{\lambda}_n^0 := \frac{1}{n} \sum_{i=1}^n \frac{N_i}{E_i}.$$

Cependant, si la variance est proportionnelle à E_i comme proposé, l'estimateur ci-dessus n'est pas optimal, il faudrait faire une moyenne pondérée avec un poids proportionnel à l'inverse de la variance de $\frac{N_i}{E_i}$. La variance de cette variable est $\frac{\lambda}{E_i}$, ce qui donne :

$$\hat{\lambda}_n^1 := \frac{\sum_{i=1}^n N_i}{\sum_{i=1}^n E_i}.$$

Pour la variance, comme

$$\mathbb{E}[(N_i - \lambda E_i)^2] = \lambda E_i,$$

on peut utiliser :

$$\hat{\lambda}_n^2 := \frac{1}{n} \sum_{i=1}^n \frac{(N_i - \hat{\lambda}_n^1 E_i)^2}{E_i}.$$

```
lambda1 <- sum(freMTPLfreq$ClaimNb)/sum(freMTPLfreq$Exposure)
lambda2 <- mean((freMTPLfreq$ClaimNb - lambda1 *
  freMTPLfreq$Exposure)^2/freMTPLfreq$Exposure)
```

On observe que $\hat{\lambda}_n^2$ est environ deux fois supérieur à $\hat{\lambda}_n^1$, ce qui contredit l'égalité $\mathbb{E}(N_i) = \text{Var}(N_i)$ attendue sous une loi de Poisson. A priori, les données présentent une sur-dispersion et ne suivent pas une loi de Poisson.

5. À l'aide de $\hat{\lambda}_n^1$, calculez les fréquences théoriques du nombre de sinistres $N_i \in \{0, 1, 2, 3, 4\}$ sous l'hypothèse $N_i \sim \mathcal{P}(\hat{\lambda}_n^1 E_i)$. Stockez-les dans un vecteur nommé **freq_poisson**. Récupérez les fréquences empiriques (calculées à la question 3) dans **freq_empirique**.

```
freq_empirique <- table(freMTPLfreq$ClaimNb)
freq_poisson <- numeric(length(freq_empirique))
names(freq_poisson) <- names(freq_empirique)
for(i in 1:length(freq_empirique))
  freq_poisson[i] <- sum(dpois(i-1, lambda = lambda1*
    freMTPLfreq$Exposure))
```

6. Représentez graphiquement **freq_empirique** et **freq_poisson** pour les comparer. Commentez les écarts observés.

```
# On remarque qu'une comparaison directe, la très grande fréquence
  en 0 écrase visuellement celles des plus grands nombres. On
  passe en log10.
plot(names(freq_poisson), log10(freq_poisson), type="b", pch=16,
  col="#34495e", xlab="", ylab="log10-observations")
lines(names(freq_empirique), log10(freq_empirique), type="b", pch
  =16, col="#2980b9")
# Cette fois-ci on voit bien la déviation importante pour les
  grandes valeurs.
```

7. En assurance, on suppose souvent l'indépendance entre la fréquence (N_i) et la gravité moyenne des sinistres ($\frac{S_i}{N_i}$ pour $N_i > 0$). Vérifiez empiriquement cette hypothèse en calculant la corrélation entre ces deux quantités.

Pour cela :

- Agrégez `freMTPLsev` par `PolicyID` pour obtenir $S_i = \text{sum}(\text{ClaimAmount})$ pour tout i tel que $N_i > 0$.
- Effectuez une jointure à gauche avec `freMTPLfreq` sur la clé `PolicyID` afin d'ajouter `ClaimNb`.
- Calculez $\frac{S_i}{N_i}$ pour $N_i > 0$, puis la corrélation avec N_i .

La syntaxe de jointure dans `data.table` est : `X[Y, on = "clé", col_dest := col_source]`.

```
freMTPLsev_gb <- freMTPLsev[, .(S_i = sum(ClaimAmount)), by =
  PolicyID]
freMTPLsev_gb[freMTPLfreq, ClaimNb := ClaimNb, on = "PolicyID"]
freMTPLsev_gb$S_i_mean <- freMTPLsev_gb$S_i/freMTPLsev_gb$ClaimNb
cor(freMTPLsev_gb$S_i_mean, freMTPLsev_gb$ClaimNb)
```

8. Passons maintenant à l'analyse des coûts des sinistres (variable `ClaimAmount` dans `freMTPLsev`). Tracez l'histogramme de leur répartition avec 100 barres (`breaks = 100`) :

- pour l'ensemble des données ;
- en tronquant au quantile 99,5 % (pour atténuer l'effet des valeurs extrêmes) ;
- en tronquant au quantile 95 %.

Commentez les particularités observées (ex. : pics anormaux).

```
hist(freMTPLsev$ClaimAmount, xlab="Montant sinistre", main="Répartition des sinistres", col="#3498db", breaks=100)
alpha <- 0.995 # Ou 0.95
hist(freMTPLsev$ClaimAmount[freMTPLsev$ClaimAmount < quantile(
  freMTPLsev$ClaimAmount, alpha)], xlab="Montant sinistre", main="Répartition des sinistres", col="#3498db", breaks=100)
# Commentaire : Un mode prononcé vers 1 200 suggère des sinistres standardisés (ex. : bris de glace avec franchise fixe). On peut regarder ce qui se passe entre 900 et 1500 avec :
hist(freMTPLsev$ClaimAmount[freMTPLsev$ClaimAmount > 900 &
  freMTPLsev$ClaimAmount < 1500 ], xlab="Montant sinistre", main="Répartition des sinistres", col="#3498db", breaks=100)
```

9. Pour analyser la gravité des sinistres en fonction des variables explicatives, ajoutez celles de `freMTPLfreq` à `freMTPLsev` via une jointure à gauche sur `PolicyID`. Attention : convertissez d'abord `PolicyID` en entier dans `freMTPLfreq` pour éviter les erreurs de type.

```
# On remarque qu'on a une erreur, car le type de PolicyID n'est pas le même dans les deux jeux de données.
freMTPLfreq$PolicyID <- as.integer(freMTPLfreq$PolicyID)
freMTPLsev[freMTPLfreq, .(CarAge, DriverAge), on = "PolicyID"]
```

10. Calculez la matrice de corrélations entre *toutes* les variables (sauf **PolicyID**) :

```
cor(freMTPLsev[, -'PolicyID'])
cor(freMTPLsev[ClaimAmount > 300000 , -'PolicyID'])
# Pas de corrélation importante apparente pour l'ensemble mais plus
  importante sur les gros sinistres
```

Statistique descriptive avec R

TP 7

Nous allons travailler sur les données de population des communes de France (millésime 2022, France hors Mayotte).

1. Rendez-vous sur la page officielle de l'Insee : <https://www.insee.fr/fr/statistiques/8581696> Téléchargez le fichier suivant : *Évolution et structure de la population en 2022 - Commune - France hors Mayotte*. Prenez les deux formats : **.xlsx** (pour explorer les noms de colonnes dans Excel/LibreOffice) et **.csv** (celui qu'on utilisera dans R).

, téléchargez les données *Évolution et structure de la population en 2022 - Commune - France hors Mayotte* en format **csv** et au format **xlsx** pour mieux les visualiser. Avec le format **xlsx**, détectez le nom de la colonne du code géographique, et celles des effectifs de la population en 2022, au total et par tranche d'âge (ce qui fait 9 colonnes). Nous créons le vecteur avec le nom des colonnes :

```
noms_col <- c("CODGEO", "P22_POP", "P22_POP0014", "P22_POP1529", "P22_POP3044", "P22_POP4559", "P22_POP6074", "P22_POP7589", "P22_POP90P")
```

2. Grâce à **data.table**, importez dans **R** les données du fichier **csv**. Vous sélectionnez à l'import les 9 colonnes choisies, ce qui évite de tout importer. Vous nommez la variable **Pop**.

3. Rendez-vous sur :

<https://www.data.gouv.fr/datasets/communes-de-france-base-des-codes-postaux/> et téléchargez les données **communes-département-région.csv** puis importez-les dans **R**. Vous nommez la variable **Info**.

4. Les colonnes **CODGEO** et **code_commune_INSEE** représente la même chose, détectez une anomalie, expliquez-là et corrigez-là.

5. On veut enrichir **Pop** avec uniquement trois colonnes d'**Info**. Faites une jointure à gauche où vous ajoutez à **Pop** les colonnes **code_département**, **latitude**, **longitude** de **Info** que vous nommez respectivement **"DEP"**, **"LAT"**, **"LONG"**.

6. Pour quelques communes, le département n'a pas été trouvé, il vaut **NA**. Pour celles-ci, affichez la taille de la population et le code géo : sont-elles négligeables ?

7. Pour Paris, Marseille et Lyon, ajoutez à la main le code du département dans **Pop**.

8. Calculez le nombre d'habitants par département, on supprimera les **NA** résiduels, et on l'enregistre dans une variable **Pop_Dep**.

9. Après avoir installé le package si nécessaire, exécutez le code suivant.

```

# Package qui permettra d'afficher la carte
library(geodata)
# Récupération de la carte au niveau des départements
fr ← gadm(country = "FRA", level = 2, path = tempdir())
# Les numéros de département sont ici
fr$CC_2
# Test d'affichage
couleurs ← rainbow(length(fr))
plot(fr, col = couleurs, border = "white", lwd = 0.6)

```

10. Une couleur peut être décomposée en quantité de rouge, de vert et de bleu entre 0 et 255. La fonction `rgb` pour une quantité de rouge `r`, de vert `v` et de bleu renvoie `b` renvoie le code couleur correspondant `rgb(r, v, b)`, et elle est vectorielle.

Construisez un vecteur de couleurs où, pour chaque taille de population `Pop_Dep$POP_DEP`, la couleur va linéairement de `col_0 ← c(236, 240, 241)` (population nulle) à `col_max ← c(44, 62, 80)` (population la plus élevée de l'ensemble des départements).

10. Affichez la carte des départements avec les couleurs correspondantes.

Statistique descriptive avec R

TP 8

Nous allons travailler sur les données `mtcars` déjà présentes dans [R](#).

1. Affichez les données et regardez la description dans l'aide des données. Pour les 11 colonnes, décrivez pour chacune s'il s'agit de variables

- **Quantitatives continues** : des variables numériques qui correspondent à une valeur mesurée et distribuées continument. Exemple : la surcafe en m2 d'un appartement.
- **Quantitatives discrètes** : des variables numériques qui correspondent à une réalité et distribuée sur des valeurs discrètes précises et souvent en petit nombre : le nombre de pièces d'un appartement.
- **Qualitatives ordinales** : des variables qualitatives où les modalités ont un ordre logique, parfois représentées numériquement mais dont le nombre n'a plus de sens numérique : le diagnostic de performance énergétique (DPE) d'un appartement.
- **Qualitatives nominales** : des variables qualitatives pour lesquelles il n'y a aucun ordre, parfois représentées numériquement. Exemple : le type de chauffage de l'appartement.

Nom	Type
<code>mpg</code>	Quantitative continue
<code>cyl</code>	Quantitative discrète
<code>disp</code>	Quantitative continue
<code>hp</code>	Quantitative continue
<code>drat</code>	Quantitative continue
<code>wt</code>	Quantitative continue
<code>qseq</code>	Quantitative continue
<code>vs</code>	Qualitative nominale
<code>am</code>	Qualitative nominale
<code>gear</code>	Qualitative ordinale
<code>carb</code>	Quantitative discrète

La variable `gear` est qualitative ordinale car, bien qu'elle prenne des valeurs entières (3, 4, 5), l'effet d'un rapport supplémentaire n'est pas constant : passer de 3 à 4 vitesses change radicalement le comportement de la voiture, alors que passer de 4 à 5 a une incidence bien moindre et contextuel.

2. Avec les fonctions `table` et `prop.table` (usage : `table(x)` et `prop.table(table(x))`), affichez les fréquences et les proportions des variables `cyl` et `am`, puis des combinaisons `cyl`, `am` en ajoutant le second en second argument de `table`.

```
table(mtcars$cyl)
prop.table(table(mtcars$cyl))
```

```
table(mtcars$am)
prop.table(mtcars$am)

table(mtcars$cyl, mtcars$am)
prop.table(table(mtcars$cyl, mtcars$am))
```

3. Affichez l'histogramme de la variable quantitative continue **hp**.

```
hist(mtcars$hp, main = "Distribution de la puissance", xlab = "
  Puissance (chevaux)", col = "#3498db")
```

4. Avec la fonction **pie** qui prend en premier argument le résultat d'un **table**, affichez le camembert de répartition des **am**. On essayera de mettre en **labels** la signification "**Automatique**" et "**Manuelle**" plutôt que 0 ou 1, et le pourcentage de répartition.

```
table_am <- table(mtcars$am)
pie(table(mtcars$am),
  labels = paste0(c("Automatique", "Manuelle"), "\n(",
    round(100 * prop.table(table_am), 1), "%)"),
  main = "Répartition des types de transmission",
  col = c("#e74c3c", "#3498db"),
  clockwise = TRUE)
```

5. Pour afficher les fréquences des modalités d'une variable qualitative, on peut utiliser un diagramme en barres avec **barplot** qui prend en premier argument le résultat d'un **table**. Affichez la répartition des **vs**.

```
barplot(table(mtcars$vs), main = "Répartition des types de moteurs"
, ylab = "Nombre de voitures", col = c("#2980b9", "#c0392b"),
  names.arg = c("En V", "En ligne"))
```

6. Pour comparer des données quantitative par groupes qualitatifs, on utilise un **boxplot**. Affichez **mpg** en fonction de **am**, la fonction **boxplot** fonctionne comme la fonction **lm** pour la formule et la mise des données.

```
boxplot(mpg ~ am, data = mtcars, main = "Consommation par type de
  transmission", ylab = "Miles par gallon", names = c("Automatique
  ", "Manuelle"), col="#bdc3c7")
```

7. Grâce à la fonction **cut**, ajouter une colonne **classehp** qui transforme la variable quantitative continue en une variable qualitative ordinale :

hp	classehp
[0, 100[	"Faible"
[100, 200[	"Moyenne"
[200, +∞[	"Forte"

```
# On pourrait le faire à la main
mtcars$classe_hp ← "Moyenne"
mtcars$classe_hp[mtcars$hp < 100] ← "Faible"
mtcars$classe_hp[mtcars$hp > 200] ← "Elevée"
# Mais on utilise ici la fonction cut qui fournit un facteur avec
  les niveaux dans le bon ordre pour la question suivante :
mtcars$classe_hp ← cut(mtcars$hp, breaks = c(0, 100, 200, Inf),
  labels = c("Faible", "Moyenne", "Elevée"), right=FALSE)
```

8. Affichez les mpg en fonction de votre nouvelle classe avec un boxplot

```
boxplot(mpg ~ classe_hp, data = mtcars, main = "Consommation par
  classe de puissance", ylab = "Miles par gallon", col="#bdc3c7",
  xlab = "Classe HP")
```